

AI Prompt

Playbook

WHAT'S INSIDE

12 MASTER PROMPTS

One per module

SYSTEM CONTEXT

Paste once, reuse always

DEBUG PROMPTS

Fix errors fast

INTEGRATION PROMPTS

Connect all modules

DEMO PROMPTS

Build the dashboard

FlowCast AI — Event-Driven Congestion System

Copy-paste prompts for every module.

Use with Claude, ChatGPT, Cursor, or GitHub Copilot.

Build the complete system prompt by prompt.

```
import pytorch_forecasting as ptf
from transformers import DistilBertClassifier
import torch_geometric.nn as gnn
model = TemporalFusionTransformer(...)
alert = lstm_autoencoder.detect(feed)
dashboard.update(crs_score=fusion(A, B))
```

MODULES

12 complete

AI TOOLS

Claude / GPT / Cursor

FORMAT

Copy-paste ready

LEVEL

Hackathon win

How To Use This Playbook

This playbook gives you **one complete, copy-paste prompt per module** of FlowCast AI. Each prompt is self-contained — it tells the AI what you are building, what the output must be, and what tech stack to use. You do not need to explain the project each time.

STEP	ACTION	DETAIL
1	Set up your AI tool	Open Claude (claude.ai), ChatGPT, or Cursor. For Cursor, open your project folder first.
2	Paste the SYSTEM CONTEXT (Prompt 0)	This is the one-time setup prompt. Paste it at the START of every new chat session. It tells the AI your full project context.
3	Paste the module prompt	Copy the prompt for the module you are building (e.g., Prompt 3 for TFT training). Paste it immediately after the system context.
4	Replace [PLACEHOLDERS]	Every prompt has [BRACKETED] placeholders. Replace them with your actual dataset path, column names, etc.
5	Review and run the code	The AI will generate complete, runnable code. Review it, run it, then use the DEBUG PROMPTS if something breaks.
6	Use integration prompts	Once individual modules work, use the Integration Prompts (Section 4) to connect them into the full pipeline.

Table of Contents

SECTION 1 SYSTEM CONTEXT PROMPTS 4

PROMPT 0 Master System Context (paste once per session) 4

SECTION 2 MODULE BUILD PROMPTS 5

PROMPT 1 Dataset Exploration & EDA 5

PROMPT 2 Feature Engineering Pipeline 6

PROMPT 3 Temporal Fusion Transformer — Planned Event Forecasting . 7

PROMPT 4 LSTM Autoencoder — Anomaly Detection 8

PROMPT 5 DistilBERT NLP — Incident Classification 9

PROMPT 6 spaCy NER — Road/Junction Extraction 10

PROMPT 7 Graph Neural Network (GAT) — Road Topology 11

PROMPT 8 Congestion Risk Score Fusion Engine 12

PROMPT 9 BTP Officer Deployment Planner 13

SECTION 3 DASHBOARD & DEMO PROMPTS 14

PROMPT 10 Streamlit Dashboard — Full 7-Tab App 14

PROMPT 11 Hugging Face Spaces Deployment 15

SECTION 4 INTEGRATION & DEBUG PROMPTS 16

PROMPT 12 Full Pipeline Integration 16

DEBUG A Fix training errors (model / data) 17

DEBUG B Fix dashboard / Streamlit errors	17
SECTION 5 SUBMISSION WRITING PROMPTS	18
PROMPT W1 Write project description for Devfolio	18
PROMPT W2 Write GitHub README	18
PROMPT W3 Generate instructions to run	18

SECTION 1 SYSTEM CONTEXT PROMPTS

Paste once at the start of every session

Paste Prompt 0 at the **very beginning** of every new chat session, before any other prompt. It gives the AI full context about your project so you do not need to explain it again.

P 0 Master System Context — Paste Once Per Session

MODULE: GLOBAL CONTEXT

COPY THIS PROMPT — paste into Claude / ChatGPT / Cursor / GitHub Copilot

```
# SYSTEM CONTEXT — Read this before every response
You are an expert AI/ML engineer helping me build FlowCast AI for the
Flipkart Gridlock 5.0 national hackathon (Round 2).
Project: FlowCast AI — Event-Driven Traffic Congestion Prediction System
Theme: Event-Driven Congestion (Planned & Unplanned)
Hackathon partner: Bengaluru Traffic Police (BTP)
What the system does:
- Predicts traffic congestion 2 hours before it forms
- Handles PLANNED events (IPL, festivals, rallies) using a Temporal
Fusion Transformer (TFT) — pytorch-forecasting library
- Handles UNPLANNED events (accidents, VIP convoys, rain) using an
LSTM Autoencoder for anomaly detection + DistilBERT NLP pipeline
- Uses a Graph Attention Network (GAT) on Bengaluru's OSMnx road graph
- Outputs to a Streamlit dashboard with a Folium map for BTP officers
Tech stack:
- Python 3.10
- pytorch-forecasting (TFT), PyTorch, PyTorch Geometric (GNN)
- HuggingFace Transformers (DistilBERT), spaCy (NER)
- Streamlit, Plotly, Folium (dashboard)
- OSMnx, NetworkX (road graph)
- Pandas, Dask, SQLite (data pipeline)
- OpenWeatherMap API, GNews API (external data)
- Deployment: Hugging Face Spaces
IMPORTANT rules you must follow:
- Always write complete, runnable Python code (no placeholders in code)
- Add inline comments explaining every major block
- Use requirements.txt-compatible library versions
- Assume dataset is a CSV at path [DATASET_PATH] with columns:
[TIMESTAMP_COL], [SEGMENT_ID_COL], [SPEED_COL], [VOLUME_COL],
[OCCUPANCY_COL], [EVENT_TYPE_COL], [VENUE_ID_COL]
- All models must run on CPU (no mandatory GPU)
- Every script must have a main() function and if __name__ == '__main__'
```

TIPS FOR THIS PROMPT

- Replace [DATASET_PATH] and all [COLUMN_NAME] placeholders with your actual CSV path and column names before pasting.
- Paste this FIRST in every new chat — then paste whichever module prompt you need next.
- In Cursor: paste this as a .cursorrules file in your project root so it applies automatically.
- If the AI forgets context mid-conversation, paste just the first 10 lines again as a reminder.

SECTION 1 MODULE BUILD PROMPTS

One prompt per module — build in order

Build each module in the order shown. Each prompt builds on the previous one.

P 1 Dataset Exploration & EDA MODULE: DATA PIPELINE

COPY THIS PROMPT — paste into Claude / ChatGPT / Cursor / GitHub Copilot

```
# PROMPT 1 — Dataset EDA and Exploration
# Paste after System Context (P0)
You are an expert data scientist. Perform complete EDA on my traffic dataset.
Dataset path: [DATASET_PATH]
File format: CSV
Write a complete Python script called `01_eda.py` that does ALL of these:
1. LOAD AND INSPECT:
  - Load the CSV with pandas
  - Print shape, dtypes, head(10), describe()
  - Check for missing values (count + percentage per column)
  - Check for duplicate rows
2. TEMPORAL ANALYSIS:
  - Parse timestamp column to datetime
  - Plot: traffic volume by hour of day (bar chart)
  - Plot: traffic speed by day of week (box plot)
  - Plot: monthly trends in congestion
  - Identify: top 5 highest congestion timestamps
3. EVENT ANALYSIS:
  - Count events by event_type (bar chart)
  - Plot: average speed DURING events vs non-event periods
  - Create a table: event_type vs avg_congestion_impact
4. ANOMALY OVERVIEW:
  - Plot: speed distribution (histogram + KDE)
  - Flag: timestamps where speed < 20th percentile (likely incidents)
  - Count anomaly periods by hour and by day
5. SEGMENT ANALYSIS:
  - Find: top 10 most congested road segments (by avg low speed)
  - Plot: heatmap of congestion by segment vs hour_of_day
  - Identify: segments that are ONLY congested during events
6. OUTPUT:
  - Save all plots to /outputs/eda/ folder as PNG files
  - Print a summary report: dataset date range, total events,
  avg normal speed, avg event speed, % data with events
  - Save a clean summary CSV: eda_summary.csv
Use: pandas, matplotlib, seaborn, plotly (for interactive charts)
Make all plots publication-quality with proper titles and axis labels.
```

TIPS FOR THIS PROMPT

- Run this FIRST before any model training. EDA output will tell you if data is good.
- Check the eda_summary.csv — if avg event speed is not lower than normal speed, your event column may be mislabelled.
- If missing values > 20% for any column, ask the AI for a follow-up imputation script.
- Share the EDA plots in your presentation — judges love seeing you actually explored the data.

P 2 Feature Engineering Pipeline

MODULE: DATA PIPELINE

COPY THIS PROMPT — paste into Claude / ChatGPT / Cursor / GitHub Copilot

```
# PROMPT 2 — Feature Engineering
# Paste after P0 (System Context)
Build a complete feature engineering pipeline for FlowCast AI.
Save as `02_features.py`
Input: cleaned traffic CSV at [DATASET_PATH]
Output: feature_matrix.parquet (ready for TFT training)
Build ALL of these feature groups:
1. TEMPORAL FEATURES (cyclic encoding to preserve periodicity):
- hour_sin = sin(2*pi*hour/24), hour_cos = cos(2*pi*hour/24)
- day_sin = sin(2*pi*day_of_week/7), day_cos = cos(2*pi*day_of_week/7)
- month_sin = sin(2*pi*month/12), month_cos = cos(2*pi*month/12)
- is_weekend (bool), is_holiday (use Indian holidays list)
- is_school_day (bool), is_monsoon_season (June-September)
2. LAG FEATURES (for each: speed, volume, occupancy):
- lag_1h: value 1 hour ago
- lag_24h: same time yesterday
- lag_168h: same time last week
- rolling_mean_1h, rolling_mean_6h, rolling_mean_24h
- rolling_std_1h (volatility proxy)
3. EVENT FEATURES:
- event_type_encoded (one-hot for: ipL, festival, rally,
concert, marathon, government, none)
- hours_to_event_start (negative = event ongoing)
- hours_since_event_end (0 if event not over)
- expected_attendance_normalized (0-1 scale)
- venue_proximity_km (haversine distance from segment to venue)
- event_phase: pre_event, during_event, post_event, no_event
4. TARGET VARIABLE:
- congestion_risk_score (CRS) = formula below:
speed_ratio = current_speed / historical_avg_speed_for_hour
volume_ratio = current_volume / road_capacity
CRS = (1 - speed_ratio) * 0.6 + volume_ratio * 0.4
CRS = clip(CRS, 0, 1) * 10 (scale to 0-10)
5. LABEL COLUMNS (for supervised tasks):
- is_congested (bool: CRS > 6.0)
- congestion_severity: low(0-4), medium(4-7), high(7-10)
- is_event_related (bool: event within 3 hours and 5 km)
6. VALIDATION:
- Print: feature correlation matrix (top 20 correlations)
- Print: features with > 5% missing after engineering
- Assert: no NaN in final feature matrix
- Save train/val/test split: 70/15/15 by TIME (not random!)
- Test set must be the LATEST 15% of data (holdout)
Add docstrings to every function. Use type hints.
```

TIPS FOR THIS PROMPT

- The 70/15/15 split MUST be chronological — never random. Traffic data has time dependencies.
- If venue_proximity_km gives errors, ask for a fallback that uses segment_id to venue_id mapping instead of GPS.
- Save as .parquet (not CSV) — much faster to load during training.
- The CRS formula is your key metric — verify it makes intuitive sense: slow + full road = high score.

P
3

Temporal Fusion Transformer — Planned Event Forecasting

MODULE: MODULE A — PLANNED EVENTS

COPY THIS PROMPT — paste into Claude / ChatGPT / Cursor / GitHub Copilot

```
# PROMPT 3 — Temporal Fusion Transformer Training
# Paste after P0 (System Context)
Build the complete TFT training pipeline for FlowCast AI.
Save as `03_tft_train.py`
Input: feature_matrix.parquet (from Prompt 2)
Output: trained TFT model saved to /models/tft_flowcast.ckpt
Use the pytorch-forecasting library (version >= 1.0.0)
COMPLETE IMPLEMENTATION REQUIRED:
1. DATA SETUP:
from pytorch_forecasting import TimeSeriesDataSet, TemporalFusionTransformer
from pytorch_forecasting.data import GroupNormalizer
from pytorch_forecasting.metrics import QuantileLoss, SMAPE
- Time index: integer sequence (minutes from dataset start)
- Group id: segment_id (each road segment is a time series)
- Target: congestion_risk_score (CRS)
- Max encoder length: 672 (7 days * 96 half-hours)
- Max prediction length: 16 (4 hours in 15-min steps)
2. VARIABLE SPECIFICATION:
Static categoricals: [segment_id, road_type]
Static reals: [road_capacity, lane_count, segment centrality]
Time-varying known reals: [hour_sin, hour_cos, day_sin, day_cos,
month_sin, month_cos, hours_to_event_start, expected_attendance_normalized,
venue_proximity_km, is_holiday, is_weekend, is_monsoon_season]
Time-varying unknown reals: [speed_lag_1h, speed_lag_24h, volume_lag_1h,
rolling_mean_1h, rolling_std_1h, CRS_lag_1h, CRS_lag_24h]
Time-varying known categoricals: [event_type_encoded, event_phase]
Target normalizer: GroupNormalizer(groups=['segment_id'])
3. MODEL CONFIG:
TemporalFusionTransformer(
hidden_size=128,
attention_head_size=4,
dropout=0.15,
hidden_continuous_size=64,
output_size=7, # 7 quantiles
loss=QuantileLoss(quantiles=[0.05,0.1,0.25,0.5,0.75,0.9,0.95]),
log_interval=10,
reduce_on_plateau_patience=5,
)
4. TRAINING:
- PyTorch Lightning Trainer
- max_epochs=100, batch_size=64
- Early stopping: monitor='val_loss', patience=10
- Learning rate finder: run before training
- Save best checkpoint by val_loss
- Log: train_loss, val_loss, SMAPE per epoch
5. EVALUATION on VALIDATION SET:
- Plot: predicted vs actual CRS (line chart, 3 sample segments)
- Plot: variable importance (from interpret() method)
- Plot: attention weights (temporal patterns learned)
- Print: SMAPE, MAE, RMSE, QuantileLoss(P10,P90)
- Print: which event_types are predicted best/worst
6. INFERENCE FUNCTION:
def predict_congestion(event_type, venue_id, event_datetime,
attendance, segment_ids, hours_ahead=4):
# Returns: dict{segment_id: [CRS_P10, CRS_P50, CRS_P90] per 15min}
pass
IMPORTANT: Add gradient clipping=0.1. Use AdamW optimizer.
```

TIPS FOR THIS PROMPT

- If pytorch-forecasting install fails: pip install pytorch-forecasting lightning
- The learning rate finder is critical — always run it. Wrong lr is the #1 cause of TFT training failure.
- If training is slow: reduce hidden_size to 64, max_encoder_length to 336 (3.5 days) for a fast baseline.
- The variable importance plot is GOLD for your presentation — judges love interpretable AI.
- If SMAPE > 25% after 50 epochs, ask for a follow-up prompt to switch to LightGBM as fallback.

**P
4****LSTM Autoencoder — Unplanned Anomaly Detection**

MODULE: MODULE B — UNPLANNED EVENTS

COPY THIS PROMPT — paste into Claude / ChatGPT / Cursor / GitHub Copilot

```
# PROMPT 4 – LSTM Autoencoder for Anomaly Detection
# Paste after P0 (System Context)
Build the LSTM Autoencoder for unplanned traffic incident detection.
Save as `04_lstm_autoencoder.py`
Input: feature_matrix.parquet (ONLY the rows labelled is_event_related=False)
Output: trained autoencoder at /models/lstm_ae.pt
threshold.json (anomaly detection thresholds per segment)
COMPLETE IMPLEMENTATION:
1. DATA PREP:
- Filter to NORMAL traffic only (exclude all event-related rows)
- Create sliding windows: window_size=6 (30 minutes of 5-min data)
- Features per timestep: [speed_normalized, volume_normalized,
occupancy_normalized] (3 features)
- Shape: (n_windows, window_size=6, n_features=3)
- Train on 80% of normal data. Val on 20%.
2. AUTOENCODER ARCHITECTURE:
class TrafficAutoencoder(nn.Module):
Encoder: LSTM(input=3, hidden=64, layers=2, dropout=0.1)
-> LSTM(hidden=64, hidden=32, layers=1)
-> Linear(32, 16) # bottleneck
Decoder: Linear(16, 32)
-> LSTM(input=32, hidden=32, layers=1)
-> LSTM(hidden=32, hidden=64, layers=2)
-> Linear(64, 3) # reconstruct input
3. TRAINING:
- Loss: MSE reconstruction loss
- Optimizer: Adam, lr=1e-3
- Epochs: 100, batch_size=128
- Early stopping on val_loss (patience=10)
- Save best model by val_loss
4. THRESHOLD CALIBRATION (PER SEGMENT):
- After training, run all NORMAL validation windows through model
- Compute reconstruction_error per window (MSE)
```



```
- Fit Gaussian to error distribution: mean, std
- Threshold = mean + 2.5 * std
- Save per-segment thresholds to threshold.json
5. INFERENCE & ALERT SYSTEM:
def detect_anomaly(segment_id, latest_30min_data):
# Returns: {is_anomaly: bool, error: float,
# threshold: float, confidence: float (0-1)}
pass
def run_realtime_monitor(data_stream, alert_callback):
# Continuously monitors all segments
# Calls alert_callback(segment_id, anomaly_dict)
# when anomaly detected for >= 2 consecutive windows
pass
6. EVALUATION on HELD-OUT TEST SET:
- Use known incident periods as ground truth
- Compute: Precision, Recall, F1, ROC-AUC
- Plot: reconstruction error distribution (normal vs incident)
- Plot: ROC curve
- Print: false positive rate at chosen threshold
Add: segment-level error heatmap (which segments are most volatile?)
```

TIPS FOR THIS PROMPT

- Train ONLY on normal data — never include incident periods. The model learns "normal" to detect "abnormal".
- Per-segment thresholds are critical — a motorway has different normal variance than a residential street.
- The 2-consecutive-window rule (10 minutes) is the key false-positive filter. Do not remove it.
- If recall < 80%: lower threshold multiplier from 2.5 to 2.0 and re-evaluate.

P 5 DistilBERT NLP — Incident Classification

MODULE: MODULE B — NLP PIPELINE

COPY THIS PROMPT — paste into Claude / ChatGPT / Cursor / GitHub Copilot

```
# PROMPT 5 — DistilBERT Incident Classification
# Paste after P0 (System Context)
Build the NLP incident classification pipeline.
Save as `05_nlp_classifier.py`
This classifies real-time news/tweets into traffic incident types.

1. DATASET CREATION (if no labelled dataset provided):
Create a synthetic training dataset of 500 examples per class:
Classes: [accident_major, accident_minor, vip_convoy,
protest_bandh, waterlogging, road_closure, clear]
Generate realistic Bengaluru traffic incident texts like:
- "Major accident on Outer Ring Road near Marathahalli. 3 vehicles involved."
- "VIP movement on MG Road. Expect heavy delays till 4 PM."
- "Waterlogging reported at Hebbal flyover. Avoid route."
Save as training_data.json [{text, label}, ...]

2. FINE-TUNING:
Base model: distilbert-base-multilingual-cased (handles Kannada-English)
from transformers import (DistilBertTokenizerFast,
DistilBertForSequenceClassification, Trainer,
TrainingArguments)
- Tokenize with max_length=128, padding=True, truncation=True
- Train: 80% data, Val: 20%
- TrainingArguments: num_epochs=5, batch=16, lr=2e-5, weight_decay=0.01
- Metric: F1-score (macro) on validation set
- Save model to /models/nlp_classifier/

3. NAMED ENTITY RECOGNITION (spaCy):
Build a custom spaCy NER model to extract from incident text:
- ROAD_NAME: "Outer Ring Road", "MG Road", "Silk Board flyover"
- JUNCTION: "Marathahalli junction", "Hebbal"
- DIRECTION: "northbound", "towards airport"
- SEVERITY: "minor", "major", "severe"
Training data: 200 annotated examples using spaCy format
Save NER model to /models/ner_model/

4. GEOCODING MODULE:
Build a Bengaluru junction gazetteer:
- Dictionary: {junction_name: (lat, lon, osmnx_node_id)}
- Include top 200 Bengaluru junctions/landmarks
- Function: geocode_incident(road_name, junction) -> osmnx_node_id

5. COMPLETE PIPELINE FUNCTION:
def classify_incident(text: str) -> dict:
# Returns:
# {
# incident_type: str, # e.g. "accident_major"
# confidence: float, # 0.0-1.0
# road_name: str, # extracted by NER
# junction: str, # extracted by NER
# severity: str, # extracted by NER
# osmnx_node_id: int, # geocoded location
# alert_level: str, # "RED"/"AMBER"/"GREEN"
# }

6. NEWS SCRAPER:
Build a scraper that polls every 60 seconds:
- GNews API (gnews library): query="Bengaluru traffic"
- Twitter/X: @BlrCityPolice timeline (use tweepy)
- Returns: list of new texts since last poll
- Deduplication by content hash
```

TIPS FOR THIS PROMPT

- If you do not have real labelled data, the synthetic dataset generation step is critical — run it first.
- distilbert-base-multilingual-cased handles code-switching (English + Kannada words in same tweet) better than English-only.
- The Bengaluru junction gazetteer is a one-time build — once done, it never needs retraining.
- For the demo, have 10 pre-saved example texts so you can show the classifier working instantly without waiting for live scraping.

P 6 Graph Neural Network — Road Topology (GAT)

MODULE: SPATIAL LAYER — GNN

COPY THIS PROMPT — paste into Claude / ChatGPT / Cursor / GitHub Copilot

```
# PROMPT 6 — Graph Attention Network on Bengaluru Road Graph
# Paste after P0 (System Context)
Build the GNN spatial layer for congestion propagation modelling.
Save as `06_gnn_model.py`
1. ROAD GRAPH CONSTRUCTION:
import osmnx as ox
import torch_geometric
- Download Bengaluru drivable road network:
G = ox.graph_from_place("Bengaluru, Karnataka, India",
network_type='drive')
- Convert to PyTorch Geometric format:
nodes: intersections (node_id, lat, lon)
edges: road segments (from_node, to_node, length, maxspeed, lanes)
- Node features: [current_CRS, historical_avg_CRS,
degree centrality, betweenness centrality,
is_near_stadium, is_near_hospital, is_on_ring_road]
- Edge features: [road_length, max_speed, lane_count,
is_one_way, signal_density]
2. GAT MODEL:
from torch_geometric.nn import GATConv, BatchNorm
class CongestionGAT(torch.nn.Module):
def __init__(self):
# Layer 1: GATConv(node_features=7, out=64, heads=4)
# Layer 2: GATConv(64*4=256, out=32, heads=4)
# Layer 3: GATConv(32*4=128, out=1, heads=1)
# BatchNorm after each layer
# Dropout=0.2 between layers
# Output: CRS prediction per node (0-10)
3. TRAINING:
- Labels: CRS at each node at time T+1, T+2, T+4, T+8 (cascade)
- Loss: MSE + spatial smoothness regularizer
(adjacent nodes should have correlated predictions)
- Optimizer: Adam lr=1e-3, epochs=200
- Split: 70/15/15 by time (same as TFT)
```

```
4. PROPAGATION SIMULATOR:
def simulate_propagation(incident_node_id, severity, time_steps=8):
# Runs GAT forward pass with incident at source node
# Returns: {node_id: [CRS_t1, CRS_t2...CRS_t8]} for all nodes
# t steps are 15-min intervals (covers 2 hours)
5. VISUALIZATION:
- Build a Folium overlay function:
generate_propagation_map(propagation_dict, timestamp)
-> returns folium HTML with colour-coded road segments
(GREEN=CRS<4, AMBER=4-7, RED=7-10)
- Animation: show congestion spreading over 8 time steps
6. INTEGRATION HOOK:
- Expose: update_graph_features(segment_id, new_CRS)
- This is called every 15 min by the fusion engine
- Returns: updated CRS predictions for all downstream segments
```

TIPS FOR THIS PROMPT

- OSMnx download takes 2-5 minutes for Bengaluru. Save the graph to a .gpkg file and reload from disk.
- If torch_geometric install fails: pip install torch_geometric torch_scatter torch_sparse
- For the demo, precompute the Bengaluru graph and store as graph.pt — do not download live during demo.
- The propagation simulator is the most impressive demo moment — show congestion spreading like a wave on the map.

P
7

Congestion Risk Score Fusion Engine

MODULE: FUSION LAYER

COPY THIS PROMPT — paste into Claude / ChatGPT / Cursor / GitHub Copilot

```
# PROMPT 7 — CRS Fusion Engine
# Paste after P0 (System Context)
Build the fusion engine that combines all model outputs.
Save as `07_fusion_engine.py`
This is the central hub — it merges Module A (TFT), Module B (LSTM AE
+ NLP), and the GNN into a single Congestion Risk Score (CRS) per
road segment per 15-minute timestep.
1. INPUT INTERFACES:
- From TFT (Module A): {segment_id: [P10, P50, P90]} for next 16 steps
- From LSTM AE (Module B): {segment_id: {is_anomaly, confidence, error}}
- From NLP (Module B): {incident_type, node_id, severity, confidence}
- From GNN: {segment_id: propagated_CRS} (after incident)
- From Weather API: {rain_mm, visibility, wind_speed}
2. FUSION LOGIC:
For each segment and each future timestep:
# Base prediction from TFT
crs_base = tft_output[segment_id]['P50'][timestep]
# Anomaly boost (if LSTM flags anomaly)
anomaly_boost = 0
if lstm_output[segment_id]['is_anomaly']:
    anomaly_boost = lstm_output[segment_id]['confidence'] * 3.0
# NLP incident boost (for nearby segments)
nlp_boost = 0
if nlp_incident and distance_to_incident < 3km:
    severity_map = {major: 4.0, minor: 2.0, convoy: 1.5, ...}
    nlp_boost = severity_map[nlp_incident.type] * (1 - dist/3.0)
# Weather multiplier
weather_mult = 1.0
if rain_mm > 10: weather_mult = 1.3
if visibility < 500m: weather_mult *= 1.2
# GNN spatial correction
gnn_crs = gnn_output.get(segment_id, crs_base)
# Final CRS (weighted combination)
crs_final = (0.5*crs_base + 0.3*gnn_crs) * weather_mult
crs_final += anomaly_boost + nlp_boost
crs_final = min(10.0, max(0.0, crs_final))
3. ALERT LEVELS:
RED = CRS >= 7.5 (deploy officers NOW)
AMBER = CRS 5.0-7.4 (monitor closely, pre-position)
GREEN = CRS < 5.0 (normal conditions)
4. ECONOMIC IMPACT CALCULATOR:
def calc_economic_impact(segment_id, crs, duration_hours):
    avg_vehicles_per_hour = get_historical_volume(segment_id)
    delay_per_vehicle_min = (crs / 10) * 30 # max 30 min delay
    bengaluru_avg_salary_per_min = 4.50 # INR
    impact_inr = (avg_vehicles_per_hour * duration_hours
    * delay_per_vehicle_min * bengaluru_avg_salary_per_min)
    return impact_inr
```

5. OFFICER DEPLOYMENT RECOMMENDER:

```
def recommend_deployment(top_n_segments, event_info):
# For each RED/AMBER segment, recommend:
# - Number of officers (based on CRS and road capacity)
# - Deployment time (before_event_start - lead_time_minutes)
# - Priority rank (1 = most critical)
# Returns list of DeploymentRecommendation objects
```

6. OUTPUT FORMAT (JSON per update cycle):

```
{
"timestamp": "2025-03-15T19:00:00",
"segments": [
{"segment_id": "MG001", "crs_P50": 8.2, "crs_P10": 7.1,
"crs_P90": 9.0, "alert_level": "RED",
"forecast_16_steps": [...], "economic_impact_inr": 45000},
...
],
"active_alerts": [...],
"deployment_brief": [...],
"total_economic_impact_inr": 1234567
}
```

Run the fusion engine every 15 minutes in a background thread.
Cache the latest output to fusion_output.json for the dashboard.

TIPS FOR THIS PROMPT

- The fusion engine is the hardest module — build and test Module A and B separately first.
- The economic impact calculator is unique — no other team will have this. Show it prominently in your demo.
- Cache fusion_output.json every 15 minutes — the Streamlit dashboard reads this file (no direct model calls from UI).
- The weather multiplier is simple but powerful — especially for Bengaluru monsoon season (June–September).

**P
8****BTP Officer Deployment Planner**

MODULE: OPERATIONAL LAYER

COPY THIS PROMPT — paste into Claude / ChatGPT / Cursor / GitHub Copilot

```
# PROMPT 8 — BTP Officer Deployment Planner
# Paste after P0 (System Context)
Build the officer deployment recommendation module.
Save as `08_deployment_planner.py`
This module converts fusion engine output into actionable BTP
officer deployment instructions.
1. INPUT:
- fusion_output.json (from Prompt 7)
- event_calendar.json: {event_id, type, venue, datetime, attendance}
- btp_stations.json: {station_id, lat, lon, officer_count, vehicles}
2. DEPLOYMENT ALGORITHM:
For each RED/AMBER segment in fusion output:
a) JUNCTION PRIORITISATION:
- Sort segments by CRS (highest first)
- Group nearby segments into "deployment zones" (< 500m apart)
```

```

- Score each zone: zone_score = max_CRS * log(vehicle_volume)
b) OFFICER COUNT FORMULA:
officers_needed = ceil(CRS * road_capacity / 500)
# Clamp: min=1, max=8 per junction
c) DEPLOYMENT TIMING:
For PLANNED events: deploy at event_start - 90 minutes
For UNPLANNED incidents: deploy immediately (within 10 minutes)
d) NEAREST STATION MATCHING:
- Find nearest BTP station to each zone (Haversine distance)
- Check officer availability (subtract already deployed)
- Assign nearest available station to each zone
3. MISSION BRIEF GENERATOR:
Generate a human-readable mission brief for BTP officers:
def generate_mission_brief(deployment_plan, event_info) -> str:
# Returns formatted text:
# "TRAFFIC ADVISORY — [DATE] [TIME]
# EVENT: IPL Match — Royal Challengers vs CSK at Chinnaswamy
# PREDICTED PEAK: 21:30 - 23:15 (CRS: 8.7/10 — CRITICAL)
#
# DEPLOYMENT INSTRUCTIONS:
# 1. M.G. Road × Residency Road — 4 officers — Deploy by 21:00
# (Priority: CRITICAL | Vehicle count: ~3,400/hr)
# 2. Cubbon Road junction — 2 officers — Deploy by 21:15
# (Priority: HIGH | Vehicle count: ~1,800/hr)
# ...
# ECONOMIC IMPACT IF UNMANAGED: █12.4 lakh/hour
# ESTIMATED REDUCTION WITH DEPLOYMENT: 35%"
4. PDF BRIEF GENERATOR:
- Use reportlab to generate a formatted PDF mission brief
- Include: BTP logo area, event details, map placeholder,
deployment table, officer signatures section
- Save to /outputs/briefs/brief_{event_id}_{date}.pdf
5. REST API ENDPOINT (FastAPI):
from fastapi import FastAPI
app = FastAPI()
@app.get("/brief/today")
def get_today_brief():
# Returns JSON + downloadable PDF link
@app.get("/alerts/live")
def get_live_alerts():
# Returns current RED/AMBER segments
@app.post("/feedback")
def submit_feedback(segment_id: str, was_accurate: bool, notes: str):
# Logs prediction accuracy for model improvement
Run with: uvicorn 08_deployment_planner:app --host 0.0.0.0 --port 8001

```

TIPS FOR THIS PROMPT

- █ The PDF mission brief is a huge differentiator in the demo — judges can literally see a BTP-ready document.
- █ `btp_stations.json` can be approximated using the actual BTP station addresses from their website.
- █ The REST API means FlowCast AI can integrate with any BTP existing system — mention this to judges.
- █ If FastAPI is new to you, simplify: just expose the brief as a Streamlit download button in the dashboard.

SECTION 9 DASHBOARD & DEMO PROMPTS

Build the Streamlit app and deploy it

P
9 Streamlit Dashboard — Full 7-Tab Application
MODULE: DASHBOARD LAYER

COPY THIS PROMPT — paste into Claude / ChatGPT / Cursor / GitHub Copilot

```
# PROMPT 9 — Streamlit Dashboard
# Paste after P0 (System Context)
Build the complete Streamlit dashboard for FlowCast AI.
Save as `dashboard/app.py`
This is the judges' window into your system. Make it impressive.

1. PAGE CONFIG:
st.set_page_config(
    page_title="FlowCast AI — BTP Dashboard",
    page_icon="■",
    layout="wide",
    initial_sidebar_state="expanded"
)
Add custom CSS for dark theme with Bengaluru BTP colour scheme.

2. SIDEBAR:
- FlowCast AI logo (text-based if no image)
- "Last updated: [timestamp]" (auto-refreshes every 60 sec)
- Mode selector: LIVE / DEMO / HISTORICAL
- Date/time selector for historical view
- Event filter: All / Planned / Unplanned

3. TAB 1 — LIVE MAP:
- Full-width Folium map of Bengaluru
- Road segments coloured by current CRS:
GREEN (CRS<4), YELLOW (4-6), ORANGE (6-7.5), RED (7.5+)
- Popup on click: segment name, current CRS, predicted peak CRS,
alert level, economic impact per hour
- Legend in top-right corner
- Auto-refresh: st.experimental_rerun() every 60 seconds

4. TAB 2 — PLANNED EVENTS (Event Forecast):
- st.dataframe: upcoming events this week (sortable)
- Click event → show 4-hour forecast chart (Plotly line chart)
with P10/P50/P90 confidence bands (shaded area)
- Bar chart: top 10 most affected road segments for selected event
- "Deployment Brief" download button (PDF from Prompt 8)

5. TAB 3 — LIVE ALERTS (Unplanned Incidents):
- Real-time alert feed (newest first)
- Each alert card shows: incident type (emoji icon), road name,
severity badge (RED/AMBER), time since detected, CRS impact
- Click alert → zoom map to incident location
- Alert history: last 24 hours table

6. TAB 4 — OFFICER BRIEF:
- "Today's Deployment Plan" auto-generated from Prompt 8
- Table: junction, officers needed, deploy_by_time, priority, station
- Download PDF brief button
- "What-if" slider: change event attendance ±50%
→ show how CRS and officer count changes in real-time

7. TAB 5 — ECONOMIC IMPACT:
- KPI cards: total_congestion_hours_today,
total_economic_impact_inr, incidents_detected,
events_managed, accuracy_last_30_days
- Time series: daily economic impact (last 30 days) — Plotly bar
- "Savings" counter: estimated impact PREVENTED by FlowCast AI
```



```

vs. no-prediction baseline
8. TAB 6 — ACCURACY LOG:
- Table: past 30 events — event_type, predicted_peak_CRS,
actual_peak_CRS, SMAPE, was_alert_correct
- Overall metrics: SMAPE, Recall (incidents detected), Precision
- Badge: "Model confidence level: HIGH/MEDIUM/LOW"
9. TAB 7 — WHAT-IF SIMULATOR:
- Dropdowns: event_type, venue, day_of_week, time, weather
- Sliders: attendance, rain_intensity
- "Run Prediction" button → show instant forecast map
- Compare: "with BTP deployment" vs "without" side by side
Load data from: fusion_output.json (updated every 15 min by background process)
Use st.cache_data with ttl=60 for all data loading functions.
Run with: streamlit run dashboard/app.py --server.port 8501

```

TIPS FOR THIS PROMPT

- Build Tab 1 (map) and Tab 2 (events) first — these are the two most impressive tabs for judges.
- Use demo mode with pre-loaded data so the dashboard works even if live APIs are unavailable.
- The What-If Simulator (Tab 7) is the most interactive — judges will play with it. Make sure it is fast.
- `st.cache_data` is essential — without caching, Streamlit reruns all model inference on every interaction.
- For the Hugging Face deployment, combine all model loading into a `@st.cache_resource` function.

P 10

Hugging Face Spaces Deployment

MODULE: DEPLOYMENT

COPY THIS PROMPT — paste into Claude / ChatGPT / Cursor / GitHub Copilot

```

# PROMPT 10 — Hugging Face Spaces Deployment
# Paste after P0 (System Context)
Generate all files needed to deploy FlowCast AI on Hugging Face Spaces.
1. REQUIREMENTS.TXT:
Generate a complete requirements.txt for the Streamlit dashboard.
Must include: streamlit, plotly, folium, streamlit-folium,
pandas, numpy, torch (CPU only: torch==2.1.0+cpu), pytorch-forecasting,
transformers, osmnx, networkx, torch_geometric, gnews, tweepy,
openweathermap, fastapi, uvicorn, reportlab, scipy, scikit-learn
Pin versions for stability.
2. README.MD FOR HF SPACES:
---
title: FlowCast AI - Traffic Congestion Predictor
emoji: 🚦
colorFrom: blue
colorTo: indigo
sdk: streamlit

```

```
sdk_version: 1.30.0
app_file: dashboard/app.py
pinned: true
---
```

[Write a 200-word description of FlowCast AI for HF Spaces page]

3. DEMO DATA GENERATOR:

Create `generate_demo_data.py` that generates realistic fake data for when live APIs are unavailable:

- 50 road segments with realistic Bengaluru names
- 7 days of simulated traffic data with 3 planned events
- 5 simulated incidents (accident, VIP, rain)
- Pre-computed `fusion_output.json` with all alert levels

This ensures the demo ALWAYS works regardless of API availability.

4. MODEL LOADING (CPU-OPTIMISED):

Create `load_models.py`:

```
@st.cache_resource
def load_all_models():
    # Load TFT from ONNX (CPU-only, no PyTorch needed at runtime)
    # Load LSTM AE weights
    # Load DistilBERT from HuggingFace (cached after first load)
    # Load GNN (pre-computed propagation matrices)
    # Load Bengaluru OSMnx graph (from saved .gpkg file)
    # Return: {tft, lstm_ae, nlp_clf, gnn, road_graph}
```

5. STARTUP SCRIPT (startup.sh):

- Download model weights from HF Hub on first run
- Generate demo data if no live connection
- Start background fusion engine thread
- Launch Streamlit on port 7860 (HF default)

6. GITHUB ACTIONS (.github/workflows/deploy.yml):

Auto-deploy to HF Spaces on push to main branch.

IMPORTANT: Make sure the demo works with ZERO external API calls (all APIs should have graceful fallbacks to demo data).

Test locally with: `HF_DEMO_MODE=true streamlit run dashboard/app.py`

TIPS FOR THIS PROMPT

- Deploy to HF Spaces AT LEAST 48 hours before submission so you have time to fix any issues.
- The demo data generator is the most important safety net — it guarantees your demo link works on submission day.
- Store model weights on HF Hub (huggingface-hub library) — not in the GitHub repo (50MB limit).
- Test in incognito mode and on mobile before submitting — judges will test both.

SECTION 11 INTEGRATION & DEBUG PROMPTS

Connect modules and fix errors fast

P 11 Full Pipeline Integration

MODULE: INTEGRATION

COPY THIS PROMPT — paste into Claude / ChatGPT / Cursor / GitHub Copilot

```
# PROMPT 11 — Full Pipeline Integration
# Paste after P0 (System Context)
Integrate all FlowCast AI modules into a single runnable pipeline.
Save as `main_pipeline.py`
This is the master script that runs the complete system.

1. PIPELINE ORCHESTRATOR:
Create FlowCastPipeline class with:
def __init__(self):
self.tft = load_tft_model('/models/tft_flowcast.ckpt')
self.lstm_ae = load_lstm_ae('/models/lstm_ae.pt')
self.nlp_clf = load_nlp_model('/models/nlp_classifier/')
self.gnn = load_gnn('/models/gnn_model.pt')
self.road_graph = load_osmnx_graph('/data/bengaluru_graph.gpkg')
self.threshold = load_json('/models/threshold.json')
def run_cycle(self, current_data: pd.DataFrame,
event_calendar: list,
news_feed: list) -> dict:
# 1. Feature engineering on current_data
# 2. Run TFT → planned forecasts
# 3. Run LSTM AE → anomaly scores
# 4. Run NLP → incident classifications
# 5. Run GNN → propagation maps
# 6. Run Fusion → final CRS per segment
# 7. Run Deployment planner → officer brief
# 8. Save to fusion_output.json
# 9. Return full output dict
# Total runtime target: < 30 seconds per cycle

2. SCHEDULED RUNNER:
import schedule, threading
def run_scheduler():
pipeline = FlowCastPipeline()
schedule.every(15).minutes.do(pipeline.run_cycle, ...)
while True:
schedule.run_pending()
time.sleep(1)

3. CLI INTERFACE:
python main_pipeline.py --mode live # run continuously
python main_pipeline.py --mode once # run one cycle
python main_pipeline.py --mode demo # use demo data
python main_pipeline.py --mode backtest # run on historical data

4. BACKTESTING MODULE:
def run_backtest(start_date, end_date):
# Replay historical data through the full pipeline
# Compare predictions vs actual CRS
# Generate backtest_report.csv with:
# timestamp, segment, predicted_CRS, actual_CRS, SMAPE, alert_correct
# Print: overall SMAPE, precision, recall by event_type

5. HEALTH CHECK:
def health_check() -> dict:
# Verify all models loaded correctly
# Run dummy inference through each model
```

```
# Check all APIs are reachable
# Return: {status: "healthy"/"degraded", details: {...}}
6. LOGGING:
- Use Python logging module (not print statements)
- Log level: INFO for normal ops, WARNING for API failures,
ERROR for model failures
- Log file: /logs/flowcast_{date}.log
- Include: timestamp, module, message, latency_ms
```

TIPS FOR THIS PROMPT

- Build the orchestrator AFTER all individual modules work. Do not skip module testing.
- The backtesting module is essential for your performance metrics slides — run it on the held-out test set.
- The health check endpoint is what shows judges the system is production-ready, not just a demo.
- Log everything — if something breaks during the judge demo, the logs will tell you why.

Debug Prompts — Paste when something breaks

DEBUG A — Fix Model / Training Errors

Paste this when your model throws an error during training:

```
I am training FlowCast AI (Event-Driven Congestion Prediction for Flipkart Gridlock 5.0). I
am getting this error: [PASTE THE FULL ERROR TRACEBACK HERE] My setup: - Script: [which
script, e.g. 03_tft_train.py] - Library versions: pytorch-forecasting=[version],
torch=[version] - Dataset shape: [rows x cols] - What I was doing when it broke: [describe
the step] Fix the error. Show me the corrected code section only (not the full script). Also
tell me WHY it broke so I understand the root cause.
```

DEBUG B — Fix Dashboard / Streamlit Errors

Paste this when Streamlit crashes or shows wrong data:

```
My Streamlit dashboard for FlowCast AI is broken. Error or problem: [PASTE ERROR MESSAGE OR
DESCRIBE WHAT IS WRONG] Code section causing the problem: [PASTE THE RELEVANT CODE BLOCK] My
data: fusion_output.json has this structure: [paste first 5 lines] Fix the issue. If it is a
data format issue, also show me how to validate fusion_output.json before the dashboard
loads it. Also add a user-friendly error message so judges see a clean "Data loading..."
screen instead of a traceback.
```

SECTION 1 SUBMISSION WRITING PROMPTS

Generate your Devfolio text, README, and run instructions

Use these prompts to generate every piece of text you need to fill the submission form. Replace [METRICS] with your actual measured numbers before submitting.

PROMPT W1 — Devfolio Project Description (3 paragraphs)

Copy this prompt and paste it into Claude or ChatGPT:

```
Write a 3-paragraph project description for FlowCast AI for the
Flipkart Gridlock 5.0 Round 2 submission form.
Context:
- Project: FlowCast AI – Event-Driven Congestion Prediction
- Theme: Event-Driven Congestion (Planned & Unplanned)
- Key metrics: SMAPE=[X]%, anomaly recall=[Y]%, NLP F1=[Z]
- Demo: [HF_SPACES_URL]
Paragraph 1 (Problem): Why Bengaluru traffic is broken and why
existing tools fail. Include a specific statistic.
Paragraph 2 (Solution): What FlowCast AI does – dual-mode, TFT +
LSTM AE + NLP. Mention the 2-hour advance warning.
Paragraph 3 (Impact): Real-world numbers – economic impact,
officer efficiency, alignment with Smart Cities Mission.
Style: confident, technical but readable. Under 300 words total.
No bullet points – flowing paragraphs only.
```

PROMPT W2 — GitHub README (Complete)

Copy this prompt and paste it into Claude or ChatGPT:

```
Write a complete GitHub README.md for FlowCast AI.
Must include sections:
1. Hero section: project name, badges (Python 3.10, License MIT,
HF Spaces link, Devfolio link)
2. Problem Statement (2 sentences)
3. Solution Overview (the 6 modules with brief desc)
4. Architecture diagram (ASCII art showing the pipeline)
5. Tech Stack (table: component, technology, purpose)
6. Performance Metrics (table with our actual results)
7. Installation (step by step: git clone, pip install, run)
8. Usage (how to run each module + the dashboard)
9. Dataset (what dataset we used, where from)
10. Demo (link to HF Spaces + GIF description placeholder)
11. Team (placeholder)
12. License (MIT)
Use proper Markdown formatting. Keep it professional.
```

PROMPT W3 — Instructions to Run (Submission Form Field)

Copy this prompt and paste it into Claude or ChatGPT:

```
Write the "Instructions to Run" text for the Flipkart Gridlock
hackathon submission form.
Requirements:
- Must work on a fresh Ubuntu 20.04 or macOS machine
- Assumes Python 3.10 installed
- Should take < 10 minutes for a judge to set up
- Must show a result (either live demo or demo mode)
Format as numbered steps. Include:
1. Clone the repo
2. Install dependencies
3. Download/place the dataset
4. Run the feature engineering script
5. Run the pipeline in DEMO mode (works without dataset)
6. Open the dashboard
7. The URL to the live HF Spaces demo (fallback)
Keep each step to one command. Flag any steps that take > 1 minute.
End with: "Alternatively, visit [HF_URL] for the live demo."
Max 250 words.
```

Build Order — Follow This Sequence

DAY 1–2	P0 + P1 + P2	System context, EDA, feature engineering
DAY 3–5	P3	TFT training (longest step — start early)
DAY 6–7	P4 + P5	LSTM Autoencoder + NLP classifier
DAY 8	P6	GNN road topology model
DAY 9	P7 + P8	Fusion engine + deployment planner
DAY 10–12	P9 + P10	Streamlit dashboard + HF Spaces deploy
DAY 13	P11	Full pipeline integration + backtesting
DAY 14–15	W1 + W2 + W3	Submission text, README, instructions
DAY 16	SUBMIT	Final check: demo link, video, slides, GitHub

"Paste Prompt 0 once. Then paste one prompt per module. The AI writes the code. You run it, fix it, ship it."

FlowCast AI — Prompt Playbook v1.0 — Flipkart Gridlock 5.0 Round 2